```
import json
import re
from datetime import datetime
import requests

class BallisticShieldDetector:

    def __init__(self, target_url: str, fraud_db: dict, timeout: int = 10):
        """
        Initializes the scanner.
        :param target_url: The target URL to investigate.
        :param fraud_db: External database of fraud signatures (IoC - Indicators of Compromise).
        """
        self.target_url = target_url.strip().rstrip("/")
        self.timeout = timeout
        self.fraud_db = fraud_db  # Injecting dynamic database into the scanner

        self.headers = {
            "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36",
            "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8"
        }

        self.findings = {
            "target_url": self.target_url,
            "scan_timestamp": datetime.utcnow().isoformat() + "Z",
            "shop_ids": [],
            "tracking_signatures": {},
            "catalog_analysis": {},
            "identity_anomalies": {},
            "risk_score": 0,
            "critical_flags": []
        }

    def fetch_url(self, path: str = "") -> tuple[int, str]:
        url = f"{self.target_url}{path}"
```

```
        try:
            response = requests.get(url, headers=self.headers, timeout=self.timeout)
            return response.status_code, response.text
        except requests.RequestException as e:
            return 0, str(e)

    def analyze_catalog_speed(self):
        """Analyzes product deployment timestamps and vendor variable leaks."""
        status_code, content = self.fetch_url("/products.json?limit=250")
        if status_code != 200:
            return

        try:
            data = json.loads(content)
            products = data.get("products", [])
            if not products:
                return

            timestamps = []
            vendors = set()

            for p in products:
                if p.get("created_at"): timestamps.append(p.get("created_at"))
                if p.get("vendor"): vendors.add(p.get("vendor").upper())

            # Heuristic: Automated deployment detection (Superhuman Speed)
            if len(timestamps) > 1:
                timestamp_counts = {t: timestamps.count(t) for t in set(timestamps)}
                max_same_second = max(timestamp_counts.values())
                if max_same_second >= 10:  # >= 10 products injected in 1 second
                    self.findings["risk_score"] += 35

self.findings["critical_flags"].append("SUPERHUMAN_SPEED_DEPLOYMENT")

            # Cross-reference leaked vendors with the dynamic database
            known_vendors = [v.upper() for v in self.fraud_db.get("known_vendor_names", [])]
            for v in vendors:
```

```
            if v in known_vendors:
                self.findings["risk_score"] += 40
                self.findings["critical_flags"].append(f"KNOWN_FRAUD_VENDOR_{v}")

    except json.JSONDecodeError:
        pass

def analyze_source_code(self):
    """Parses the DOM and cross-references with the dynamic IoC signature database."""
    status_code, html = self.fetch_url("/")
    if status_code != 200:
        return

    # 1. Extract SaaS Shop ID and cross-reference with the database
    shop_id_patterns = [
        r'Shopify\.shop\s*=\s*["\']([^"\']+)["\']',
        r'"shopId":\s*(\d+)',
        r'myshopify\.com\/(\d+)'
    ]
    found_shop_ids = set()
    for pattern in shop_id_patterns:
        found_shop_ids.update(re.findall(pattern, html))

    if found_shop_ids:
        self.findings["shop_ids"] = list(found_shop_ids)
        known_shop_ids = set(self.fraud_db.get("known_shop_ids", []))

        if found_shop_ids.intersection(known_shop_ids):
            self.findings["risk_score"] += 50
            self.findings["critical_flags"].append("MATCH_KNOWN_CRIMINAL_SHOP_ID")

    # 2. Check dynamic tracking identifiers
    clarity_match = re.search(r'clarity\.ms\/tag\/([a-zA-Z0-9]+)', html)
    if clarity_match:
        clarity_id = clarity_match.group(1)
        self.findings["tracking_signatures"]["microsoft_clarity"] = clarity_id
```

```
            # Cross-reference with our blacklist
            if clarity_id in self.fraud_db.get("known_clarity_ids", []):
                self.findings["risk_score"] += 45
                self.findings["critical_flags"].append("MATCH_BLACKLISTED_TRACKER_ID")

        # 3. Dynamic scanning for text anomalies / fake identities
        # The algorithm iterates through a list of stolen registry numbers provided by the bank
        for registry_id in self.fraud_db.get("stolen_registry_numbers", []):
            if registry_id in html:
                self.findings["risk_score"] += 40

self.findings["critical_flags"].append(f"STOLEN_IDENTITY_DETECTED:_{registry_id}")

        # The algorithm iterates through specific code errors (e.g., pasted session codes instead of TAX IDs)
        for anomaly_string in self.fraud_db.get("corrupted_identity_strings", []):
            if anomaly_string in html:
                self.findings["risk_score"] += 40

self.findings["critical_flags"].append(f"KNOWN_CODE_ANOMALY_DETECTED")

    def run_full_scan(self) -> dict:
        self.analyze_source_code()
        self.analyze_catalog_speed()
        self.findings["risk_score"] = min(self.findings["risk_score"], 100)
        return self.findings



# ========================================
# DEPLOYMENT / USAGE EXAMPLE FOR A BANK
# ========================================
if __name__ == "__main__":

    # 1. LOAD THE DATABASE (In production, fetched via API or a JSON file)
    # Here, the system administrator (e.g., SOC analyst) defines what to look for
    FRAUD_DATABASE = {
        "known_shop_ids": [
            "Enter_Compromised_Shop_ID_1",
```

```
            "Enter_Compromised_Shop_ID_2"
        ],
        "known_clarity_ids": [
            "Enter_Blacklisted_Clarity_Project_1"
        ],
        "known_vendor_names": [
            "Enter_Fake_Wholesaler_Name_1"
        ],
        "stolen_registry_numbers": [
            "Enter_Stolen_TAX_ID_or_Company_Number",
            "Enter_Stolen_KvK_Number"
        ],
        "corrupted_identity_strings": [
            "Enter_Corrupted_Code_Pasted_by_Fraudster_as_Registry_ID"
        ]
    }

    # 2. RUN THE SCANNER ON THE TARGET STORE
    target_website = "https://example-new-store-to-audit.com"

    detector = BallisticShieldDetector(target_url=target_website,
fraud_db=FRAUD_DATABASE)
    report = detector.run_full_scan()

    print(json.dumps(report, indent=2, ensure_ascii=False))
```

---

```
import json
import re
import socket
import urllib.parse
from datetime import datetime
import requests

try:
    import dns.resolver
```

```
    DNS_MODULE_AVAILABLE = True
except ImportError:
    DNS_MODULE_AVAILABLE = False



class AdvancedBallisticShieldEngine:

    def __init__(
        self,
        target_url: str,
        fraud_db: dict,
        publicwww_api_key: str = None,
        timeout: int = 10,
    ):
        """Advanced ballistic correlation and infrastructure threat intelligence engine
(OSINT/TECHINT).

        :param target_url: Target URL of the storefront to audit.
        :param fraud_db: External database containing IoC feeds and threat
        signatures.
        :param publicwww_api_key: Optional API key for the PublicWWW source code
        search engine.
        :param timeout: Network request timeout in seconds.
        """
        self.target_url = target_url.strip().rstrip("/")
        self.parsed_url = urllib.parse.urlparse(self.target_url)
        self.domain = self.parsed_url.netloc or self.parsed_url.path
        self.domain = self.domain.split(":")[0]  # Strip port if present
        self.timeout = timeout
        self.fraud_db = fraud_db
        self.publicwww_api_key = publicwww_api_key

        self.headers = {
            "User-Agent": (
                "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36"
                " (KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36"
            ),
```

```
        "Accept": (
            "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8"
        ),
    }

    self.findings = {
        "target_url": self.target_url,
        "target_domain": self.domain,
        "scan_timestamp": datetime.utcnow().isoformat() + "Z",
        "extracted_signatures": {
            "shop_ids": [],
            "clarity_ids": [],
            "tracking_pixels": [],
            "injected_identities": [],
        },
        "external_intelligence": {
            "dns_mx_infrastructure": {},
            "wayback_domain_dropping": {},
            "code_search_clustering": {},
            "semantic_gap_audit": {},
        },
        "catalog_analysis": {},
        "risk_score": 0,
        "critical_flags": [],
    }

def _http_get(self, url: str) -> tuple[int, str]:
    """Fetches the content of the specified URL."""
    try:
        response = requests.get(
            url, headers=self.headers, timeout=self.timeout
        )
        return response.status_code, response.text
    except requests.RequestException as exc:
        return 0, str(exc)

#
```

```
# ============================================================================
    # 1. DOM AND SAAS LAYER INSPECTION (DOM / JSON ENDPOINTS)
    # ============================================================================

    def inspect_dom_and_saas_artifacts(self):
        """Extracts artifacts from the DOM and audits public SaaS endpoints."""
        status_code, html = self._http_get(self.target_url)
        if status_code != 200:
            self.findings["critical_flags"].append(
                f"STORE_UNREACHABLE_HTTP_{status_code}"
            )
            return

        # Extract SaaS Shop ID
        shop_id_patterns = [
            r'Shopify\.shop\s*=\s*["\']([^"\']+)["\']',
            r'"shopId":\s*(\d+)',
            r'myshopify\.com\/(\d+)',
            r'\/cdn\.shopify\.com\/s\/files\/1\/(\d+)\/',
        ]
        found_shop_ids = set()
        for pattern in shop_id_patterns:
            matches = re.findall(pattern, html)
            for match in matches:
                found_shop_ids.add(str(match))

        if found_shop_ids:
            self.findings["extracted_signatures"]["shop_ids"] = list(
                found_shop_ids
            )
            known_ids = set(self.fraud_db.get("known_shop_ids", []))
            if found_shop_ids.intersection(known_ids):
                self.findings["risk_score"] += 45
                self.findings["critical_flags"].append(
                    "MATCH_CRIMINAL_SHOP_ID_INFRASTRUCTURE"
```

```
        )

# Extract Tracking Project ID (e.g., Clarity)
clarity_match = re.search(
    r'clarity\.ms\/tag\/([a-zA-Z0-9]+)|clarity\s*,\s*["\']([a-zA-Z0-9]+)["\']',
    html,
)
if clarity_match:
    clarity_id = clarity_match.group(1) or clarity_match.group(2)
    self.findings["extracted_signatures"]["clarity_ids"].append(
        clarity_id
    )
    if clarity_id in self.fraud_db.get("known_clarity_ids", []):
        self.findings["risk_score"] += 40
        self.findings["critical_flags"].append(
            f"MATCH_BLACKLISTED_CLARITY_TAG_{clarity_id}"
        )

# Extract identity anomalies (e.g., corrupted TAX/company IDs)
for anomaly in self.fraud_db.get("corrupted_identity_strings", []):
    if anomaly in html:
        self.findings["extracted_signatures"][
            "injected_identities"
        ].append(anomaly)
        self.findings["risk_score"] += 35
        self.findings["critical_flags"].append(
            f"CORRUPTED_IDENTITY_ARTIFACT_DETECTED_{anomaly}"
        )

for reg_id in self.fraud_db.get("stolen_registry_numbers", []):
    if reg_id in html:
        self.findings["extracted_signatures"][
            "injected_identities"
        ].append(reg_id)
        self.findings["risk_score"] += 35
        self.findings["critical_flags"].append(
            f"HIJACKED_LEGAL_ENTITY_NUMBER_{reg_id}"
```

```
        )

# Inspect JSON catalog endpoint for Superhuman Speed & Vendor Leaks
status_json, content_json = self._http_get(
    f"{self.target_url}/products.json?limit=250"
)
if status_json == 200:
    try:
        data = json.loads(content_json)
        products = data.get("products", [])
        timestamps = [
            p.get("created_at") for p in products if p.get("created_at")
        ]
        vendors = {
            p.get("vendor").upper()
            for p in products
            if p.get("vendor")
        }

        max_per_sec = 0
        if timestamps:
            counts = {t: timestamps.count(t) for t in set(timestamps)}
            max_per_sec = max(counts.values())

        is_automated = max_per_sec >= 10
        if is_automated:
            self.findings["risk_score"] += 30
            self.findings["critical_flags"].append(
                "SUPERHUMAN_SPEED_DEPLOYMENT"
            )

        known_vendors = [
            v.upper()
            for v in self.fraud_db.get("known_vendor_names", [])
        ]
        leaked_matches = vendors.intersection(set(known_vendors))
        if leaked_matches:
```

```
                self.findings["risk_score"] += 35
                self.findings["critical_flags"].append(
                    f"LEAKED_FRAUD_VENDOR_VARIABLES_{list(leaked_matches)}"
                )

            self.findings["catalog_analysis"] = {
                "total_scanned": len(products),
                "max_products_single_second": max_per_sec,
                "automated_upload": is_automated,
                "vendors_found": list(vendors),
            }
        except json.JSONDecodeError:
            pass

    # ================================================================================
    # 2. DNS AND MAIL INFRASTRUCTURE RECONNAISSANCE (MX / A RECORDS)
    # ================================================================================

    def query_dns_and_mail_infrastructure(self):
        """Analyzes MX and DNS records for shared infrastructure footprints."""
        mx_records = []
        ip_addresses = []

        if DNS_MODULE_AVAILABLE:
            try:
                answers = dns.resolver.resolve(self.domain, "MX")
                for rdata in answers:
                    mx_records.append(str(rdata.exchange).rstrip("."))
            except Exception:
                pass

            try:
                a_answers = dns.resolver.resolve(self.domain, "A")
                for rdata in a_answers:
```

```
                ip_addresses.append(str(rdata))
            except Exception:
                pass
        else:
            # Fallback if dnspython is not installed
            try:
                ip = socket.gethostbyname(self.domain)
                ip_addresses.append(ip)
            except socket.gaierror:
                pass

        self.findings["external_intelligence"]["dns_mx_infrastructure"] = {
            "resolved_ips": ip_addresses,
            "mx_servers": mx_records,
        }

        # Cross-reference shared MX infrastructure against monitored list
        flagged_mx = self.fraud_db.get("monitored_mx_providers", [])
        for mx in mx_records:
            for pattern in flagged_mx:
                if pattern in mx:
                    self.findings["risk_score"] += 20
                    self.findings["critical_flags"].append(
                        f"SHARED_MAIL_INFRASTRUCTURE_CLUSTER_{mx}"
                    )

    # ================================================================
    # 3. WAYBACK MACHINE RECONNAISSANCE (DOMAIN DROPPING DETECTOR)
    # ================================================================

    def query_wayback_machine_api(self):
        """Queries the Wayback Machine API to detect domain dropping and recycling."""
        api_url = f"https://archive.org/wayback/available?
url={self.domain}&timestamp=20200101"
```

```
        status, response_text = self._http_get(api_url)

        if status != 200:
            self.findings["external_intelligence"][
                "wayback_domain_dropping"
            ] = {"status": "UNAVAILABLE"}
            return

        try:
            data = json.loads(response_text)
            snapshots = data.get("archived_snapshots", {})

            if "closest" in snapshots:
                first_seen = snapshots["closest"].get("timestamp")
                is_available = snapshots["closest"].get("available")

                initial_year = int(first_seen[:4]) if first_seen else None
                current_year = datetime.utcnow().year

                # Heuristic: flagged if domain existed >= 3 years ago and suddenly reappeared
as e-commerce
                is_recycled = (
                    initial_year and (current_year - initial_year >= 3)
                )

                if is_recycled:
                    self.findings["risk_score"] += 25
                    self.findings["critical_flags"].append(

f"DOMAIN_DROPPING_RECYCLED_HISTORIC_DOMAIN_FIRST_SEEN_{initial_year}"
                    )

                self.findings["external_intelligence"][
                    "wayback_domain_dropping"
                ] = {
                    "historic_snapshot_found": is_available,
                    "first_archive_timestamp": first_seen,
```

```
                "recycled_domain_flag": is_recycled,
            }
        else:
            self.findings["external_intelligence"][
                "wayback_domain_dropping"
            ] = {
                "historic_snapshot_found": False,
                "recycled_domain_flag": False,
            }
    except json.JSONDecodeError:
        pass

# ================================================================================
# 4. WEB SOURCE CODE SEARCH (PUBLICWWW / NERDYDATA CLUSTERING)
# ================================================================================

def query_publicwww_code_clustering(self):
    """Queries PublicWWW API to find external websites sharing identical tracking IDs."""
    clarity_ids = self.findings["extracted_signatures"]["clarity_ids"]
    if not clarity_ids:
        return

    target_clarity = clarity_ids[0]

    if self.publicwww_api_key:
        query = urllib.parse.quote(f'"{target_clarity}"')
        api_endpoint = f"https://publicwww.com/websites/{query}/?key={self.publicwww_api_key}&snippets=0&json=1"
        status, text = self._http_get(api_endpoint)

        if status == 200:
            try:
                data = json.loads(text)
                linked_domains = [
```

```
                    item.get("url")
                    for item in data
                    if isinstance(item, dict)
                ]
                self.findings["external_intelligence"][
                    "code_search_clustering"
                ] = {
                    "queried_signature": target_clarity,
                    "co_located_sites_count": len(linked_domains),
                    "cluster_sample": linked_domains[:5],
                }
                if len(linked_domains) > 1:
                    self.findings["risk_score"] += 30
                    self.findings["critical_flags"].append(
f"NETWORK_LINKAGE_CODE_CLUSTER_CONFIRMED_{len(linked_domains)}_SITES"
                    )
                return
            except json.JSONDecodeError:
                pass

        # Fallback audit trace if no API key is configured
        self.findings["external_intelligence"]["code_search_clustering"] = {
            "queried_signature": target_clarity,
            "query_syntax": f'https://publicwww.com/websites/"{target_clarity}"/',
            "status": "API_KEY_NOT_CONFIGURED_MANUAL_AUDIT_REQUIRED",
        }

    # ==========================================================================
    # 5. SEMANTIC GAP VERIFICATION (REGISTRY VS. MERCHANDISE DISCREPANCY)
    # ==========================================================================

    def audit_semantic_gap(self):
```

```
        """Cross-references official corporate purpose codes with actual storefront activity."""
        injected = self.findings["extracted_signatures"]["injected_identities"]
        detected_vendors = self.findings["catalog_analysis"].get(
            "vendors_found", []
        )

        # Detect non-profit/welfare identity exploitation
        # The specific ID has been replaced with a generic logic check based on the IoC
database
        stolen_registry_numbers = self.fraud_db.get("stolen_registry_numbers", [])

        if any(str(x) in injected for x in stolen_registry_numbers):
            # Placeholder for mapping logic (e.g., querying external registry API)
            registered_nace_profile = "SOCIAL_WORK_OR_NON_PROFIT_ACTIVITIES"

            is_selling_goods = len(detected_vendors) > 0 or bool(
                self.findings["catalog_analysis"]
            )

            if is_selling_goods:
                gap_severity = "CRITICAL_MISMATCH"
                self.findings["risk_score"] += 35
                self.findings["critical_flags"].append(
"SEMANTIC_GAP_DETECTED_REGISTERED_PURPOSE_VS_COMMERCIAL_RETAIL
"
                )

                self.findings["external_intelligence"]["semantic_gap_audit"] = {
                    "registered_entity_code": "MATCHED_HIJACKED_REGISTRY_ID",
                    "official_purpose": registered_nace_profile,
                    "observed_site_activity": "COMMERCIAL_E_COMMERCE_RETAIL",
                    "semantic_gap_verdict": gap_severity,
                }

    #
==========================================================================
=====
```

```
    # FULL AUDIT PIPELINE EXECUTION
    #
================================================================================

    def execute_pipeline(self) -> dict:
        """Executes all detection modules and calculates deterministic risk scoring."""
        self.inspect_dom_and_saas_artifacts()
        self.query_dns_and_mail_infrastructure()
        self.query_wayback_machine_api()
        self.query_publicwww_code_clustering()
        self.audit_semantic_gap()

        # Cap Risk Score between 0 and 100
        self.findings["risk_score"] = min(self.findings["risk_score"], 100)

        # Determine operational verdict and mitigation action
        score = self.findings["risk_score"]
        if score >= 70:
            self.findings["mitigation_action"] = {
                "verdict": "CRITICAL_FRAUD_NETWORK_CONFIRMED",
                "recommended_action": (
                    "FREEZE_PAYMENT_GATEWAY_SETTLEMENT_IMMEDIATELY"
                ),
                "reporting_obligation": "SUBMIT_SAR_STR_TO_AML_FIU",
            }
        elif score >= 40:
            self.findings["mitigation_action"] = {
                "verdict": "ELEVATED_SUSPICION_ANOMALY",
                "recommended_action": "ENFORCE_MANUAL_KYB_TIER2_INSPECTION",
                "reporting_obligation": "FLAG_FOR_TRANSACTION_MONITORING",
            }
        else:
            self.findings["mitigation_action"] = {
                "verdict": "LOW_RISK",
                "recommended_action": "PERMIT_STANDARD_PROCESSING",
                "reporting_obligation": "NONE",
```

```
        }

        return self.findings


# ================================================================================
# USAGE EXAMPLE WITH DYNAMIC IOC SIGNATURE FEEDS
# ================================================================================
if __name__ == "__main__":

    FRAUD_THREAT_DATABASE = {
        "known_shop_ids": ["ENTER_KNOWN_FRAUD_SHOP_ID_1", "ENTER_KNOWN_FRAUD_SHOP_ID_2"],
        "known_clarity_ids": ["ENTER_BLACKLISTED_CLARITY_TAG"],
        "known_vendor_names": ["ENTER_LEAKED_VENDOR_NAME"],
        "stolen_registry_numbers": ["ENTER_HIJACKED_REGISTRY_NUMBER"],
        "corrupted_identity_strings": ["ENTER_CORRUPTED_CODE_STRING"],
        "monitored_mx_providers": ["enter-suspicious-mx-domain.com"],
    }

    SUSPICIOUS_STORE = "https://enter-target-store-url-here.com"

    engine = AdvancedBallisticShieldEngine(
        target_url=SUSPICIOUS_STORE,
        fraud_db=FRAUD_THREAT_DATABASE,
        publicwww_api_key=None,
        timeout=8,
    )

    audit_result = engine.execute_pipeline()
    print(json.dumps(audit_result, indent=2, ensure_ascii=False))
```